

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего
образования
«Национальный исследовательский университет ИТМО»
Факультет Программной инженерии и компьютерной техники

Лабораторная работа №4
по дисциплине «Основы программной инженерии»
Вариант: 153

Преподаватель:
Лазеев Сергей Максимович
Выполнил:
Бармичев Григорий Андреевич
Группа: P3210

ОГЛАВЛЕНИЕ

ОГЛАВЛЕНИЕ	2
Задание	3
Исходный код разработанных MBean-классов.....	3
JConsole	7
VisualVM.....	8
Проблема.....	10
Вывод	14

Задание

1. Для своей программы из [лабораторной работы #3](#) по дисциплине "Веб-программирование" реализовать:

- MBean, считающий общее число установленных пользователем точек, а также число точек, не попадающих в область. В случае, если количество установленных пользователем точек стало кратно 15, разработанный MBean должен отправлять оповещение об этом событии.
- MBean, определяющий площадь получившейся фигуры.

2. С помощью утилиты JConsole провести мониторинг программы:

- Снять показания MBean-классов, разработанных в ходе выполнения задания 1.
- Определить время (в мс), прошедшее с момента запуска виртуальной машины.

3. С помощью утилиты VisualVM провести мониторинг и профилирование программы:

- Снять график изменения показаний MBean-классов, разработанных в ходе выполнения задания 1, с течением времени.
- Определить имя потока, потребляющего наибольший процент времени CPU.

4. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в [программе](#). По результатам локализации и устранения проблемы необходимо составить отчёт, в котором должна содержаться следующая информация:

- Описание выявленной проблемы.
- Описание путей устранения выявленной проблемы.
- Подробное (со скриншотами) описание алгоритма действий, который позволил выявить и локализовать проблему.

Исходный код разработанных MBean-классов

FigureAreaMBean:

```
package web.mbeans;

public interface FigureAreaMBean {double getR(); double getArea();}
```

PointStatsMBean:

```
package web.mbeans;

public interface PointStatsMBean {
    int getTotalPoints();
    int getMissedPoints();
    void reset();
}
```

FigureArea:

```
package web.mbeans;

public class FigureArea implements FigureAreaMBean {
    private volatile double r = 2.0;

    public void updateR(double r) {this.r = r;}

    @Override
    public double getR() {return r;}

    @Override
    public double getArea() {return r * r * (Math.PI + 10.0) / 16.0;}
}
```

MBeanRegistry:

```
package web.mbeans;

import javax.management.MBeanServer;
import javax.management.ObjectName;
import java.lang.management.ManagementFactory;

public final class MBeanRegistry {
    private static final String DOMAIN = "web.lab4";

    private MBeanRegistry() {}

    public static ObjectName register(Object bean, String name) {
        try {
            MBeanServer server = ManagementFactory.getPlatformMBeanServer();
            ObjectName objectName = new ObjectName(DOMAIN + ":type=" +
            bean.getClass().getSimpleName() + ",name=" + name);

            if (server.isRegistered(objectName)) {
                server.unregisterMBean(objectName);
            }
            server.registerMBean(bean, objectName);
            return objectName;
        } catch (Exception e) {throw new IllegalStateException("Cannot register MBean " + name, e);}
    }

    public static void unregister(ObjectName objectName) {
        if (objectName == null) {
            return;
        }

        try {
            MBeanServer server = ManagementFactory.getPlatformMBeanServer();
            if (server.isRegistered(objectName)) {
                server.unregisterMBean(objectName);
            } catch (Exception ignored) {}
        }
    }
}
```

```
}
```

MonitoringService:

```
package web.mbeans;

import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import jakarta.enterprise.context.ApplicationScoped;
import web.models.HitResult;

import javax.management.ObjectName;
import java.io.Serializable;

@ApplicationScoped
public class MonitoringService implements Serializable {
    private static final long serialVersionUID = 1L;

    private final PointStats pointStats = new PointStats();
    private final FigureArea figureArea = new FigureArea();

    private ObjectName pointStatsName;
    private ObjectName figureAreaName;

    @PostConstruct
    public void init() {
        pointStatsName = MBeanRegistry.register(pointStats, "pointStats");
        figureAreaName = MBeanRegistry.register(figureArea, "figureArea");
    }

    @PreDestroy
    public void destroy() {
        MBeanRegistry.unregister(pointStatsName);
        MBeanRegistry.unregister(figureAreaName);
    }

    public void onPointAdded(HitResult result) {
        if (result == null) {
            return;
        }

        pointStats.addPoint(result.getHit());

        if (result.getR() != null) {
            figureArea.updateR(result.getR().doubleValue());
        }
    }

    public void resetPointStats() {
        pointStats.reset();
    }
}
```

PointStats:

```
package web.mbeans;

import javax.management.MBeanNotificationInfo;
import javax.management.Notification;
import javax.management.NotificationBroadcasterSupport;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;

public class PointStats extends NotificationBroadcasterSupport implements PointStatsMBean {
    public static final String POINTS_MULTIPLE_OF_15_NOTIFICATION =
        "web.lab4.points.multiple-of-15";

    private final AtomicInteger totalPoints = new AtomicInteger();
    private final AtomicInteger missedPoints = new AtomicInteger();
    private final AtomicLong sequenceNumber = new AtomicLong();

    public void addPoint(boolean hit) {
        int total = totalPoints.incrementAndGet();

        if (!hit) {missedPoints.incrementAndGet();}

        if (total % 15 == 0) {
            sendNotification(new Notification(
                POINTS_MULTIPLE_OF_15_NOTIFICATION,
                this,
                sequenceNumber.incrementAndGet(),
                System.currentTimeMillis(),
                "Количество установленных точек стало кратно 15: " + total
            ));
        }
    }

    @Override
    public int getTotalPoints() {
        return totalPoints.get();
    }

    @Override
    public int getMissedPoints() {
        return missedPoints.get();
    }

    @Override
    public void reset() {
        totalPoints.set(0);
        missedPoints.set(0);
    }

    @Override
    public MBeanNotificationInfo[] getNotificationInfo() {
        String[] types = {POINTS_MULTIPLE_OF_15_NOTIFICATION};
        String name = Notification.class.getName();
    }
}
```

```
String description = "Уведомление о том, что количество установленных точек стало кратно 15";
return new MBeanNotificationInfo[]{new MBeanNotificationInfo(types, name, description)};
}
}
```

JConsole

MBean, считающий количество точек и промахов.

Скриншот 1: Начальное состояние

Последняя проверка: (x=-2.059, y=1.075, r=3.25) → **МИМО**

Всё	X	Y	R	Попадание	Время	Время с
▶	-2.059	1.075	3.25	Нет	2026-05-17T19:11:...	
▶	-1.709	0.975	4	Да	2026-05-17T19:10:...	
▶	-0.684	0.65	3	Да	2026-05-17T19:07:...	
▶	0.566	2.175	3	Нет	2026-05-17T19:07:...	

Скриншот 2: После попадания

Последняя проверка: (x=0, y=-1, r=3.25) → **ПОПАДАНИЕ**

Всё	X	Y	R	Попадание	Время	Время с
▶	0	-1	3.25	Да	2026-05-17T19:15:...	
▶	-2.059	1.075	3.25	Нет	2026-05-17T19:11:...	
▶	-1.709	0.975	4	Да	2026-05-17T19:10:...	
▶	-0.684	0.65	3	Да	2026-05-17T19:07:...	

Скриншот 3: После промаха

Последняя проверка: (x=2, y=-1, r=3.25) → **МИМО**

Всё	X	Y	R	Попадание	Время	Время с
▶	2	-1	3.25	Нет	2026-05-17T19:16:...	
▶	0	-1	3.25	Да	2026-05-17T19:15:...	
▶	-2.059	1.075	3.25	Нет	2026-05-17T19:11:...	
▶	-1.709	0.975	4	Да	2026-05-17T19:10:...	

Состояние MBean в JConsole:

Name	Value
MissedPoints	2
TotalPoints	4

Проверка уведомлений.

Overview	Memory	Threads	Classes	VM Summary	MBeans	Wildfly CLI
Notification buffer						
TimeStamp	Type	UserData	SeqNum	Message	Event	Source
22:22:49:773	web.lab4.points.multiple-of-15		7	Количество установленных точек стало кратно 15: 105	javax.management.Notification[source=web.lab4.type=PointStats,name=pointStats][type=web.lab4.points.multiple-of-15]	web.lab4.type=PointStats,name=pointStats
22:22:39:505	web.lab4.points.multiple-of-15		6	Количество установленных точек стало кратно 15: 90	javax.management.Notification[source=web.lab4.type=PointStats,name=pointStats][type=web.lab4.points.multiple-of-15]	web.lab4.type=PointStats,name=pointStats
22:22:29:866	web.lab4.points.multiple-of-15		5	Количество установленных точек стало кратно 15: 75	javax.management.Notification[source=web.lab4.type=PointStats,name=pointStats][type=web.lab4.points.multiple-of-15]	web.lab4.type=PointStats,name=pointStats
22:22:25:648	web.lab4.points.multiple-of-15		4	Количество установленных точек стало кратно 15: 60	javax.management.Notification[source=web.lab4.type=PointStats,name=pointStats][type=web.lab4.points.multiple-of-15]	web.lab4.type=PointStats,name=pointStats
22:22:18:833	web.lab4.points.multiple-of-15		3	Количество установленных точек стало кратно 15: 45	javax.management.Notification[source=web.lab4.type=PointStats,name=pointStats][type=web.lab4.points.multiple-of-15]	web.lab4.type=PointStats,name=pointStats
22:22:12:609	web.lab4.points.multiple-of-15		2	Количество установленных точек стало кратно 15: 30	javax.management.Notification[source=web.lab4.type=PointStats,name=pointStats][type=web.lab4.points.multiple-of-15]	web.lab4.type=PointStats,name=pointStats
22:19:09:717	web.lab4.points.multiple-of-15		1	Количество установленных точек стало кратно 15: 15	javax.management.Notification[source=web.lab4.type=PointStats,name=pointStats][type=web.lab4.points.multiple-of-15]	web.lab4.type=PointStats,name=pointStats

MBean, определяющий площадь получившейся фигуры.

Бармичев Григорий Андреевич — R3210

Веб-программирование — Лабораторная работа #3 — Вариант 11

На стартовую страницу

X:

-2

-1.5

-1

-0.5

0

0.5

1

1.5

2

Y: 0.975

R:

R = 4

Проверить точку

Очистить историю

Последняя проверка: (x=-1.709, y=0.975, r=4) → **ПОПАДАНИЕ**

Все	X	Y	R	Попадание	Время	Время с
▶	-1.709	0.975	4	Да	2026-05-17T19:10:...	
▶	-0.684	0.65	3	Да	2026-05-17T19:07:...	
▶	0.566	2.175	3	Нет	2026-05-17T19:07:...	

Страница 1 из 1

Java Monitoring & Management Console - service:jmxremote+http://localhost:9990

OverviewMemoryThreadsClassesVM SummaryMBeansWildfly CLI

Attribute values

Name	Value
Area	13.4159263389793
R	4.0

Refresh

Бармичев Григорий Андреевич — R3210

Веб-программирование — Лабораторная работа #3 — Вариант 11

На стартовую страницу

X:

-2

-1.5

-1

-0.5

0

0.5

1

1.5

2

Y: 1.075

R:

R = 3.25

Проверить точку

Очистить историю

Последняя проверка: (x=-2.059, y=1.075, r=3.25) → **МИМО**

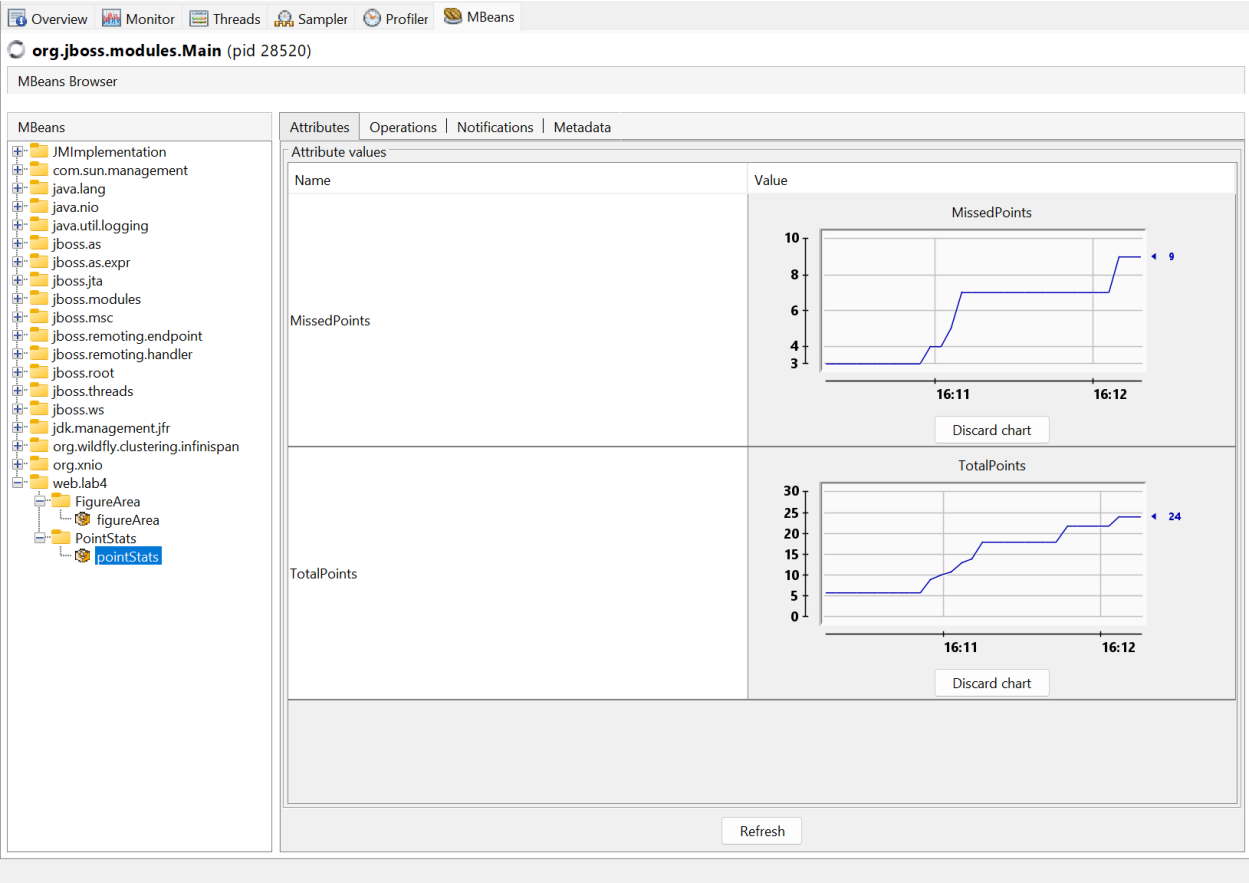
Все	X	Y	R	Попадание	Время	Время с
▶	-2.059	1.075	3.25	Нет	2026-05-17T19:11:...	
▶	-1.709	0.975	4	Да	2026-05-17T19:10:...	
▶	-0.684	0.65	3	Да	2026-05-17T19:07:...	
▶	0.566	2.175	3	Нет	2026-05-17T19:07:...	

Refresh

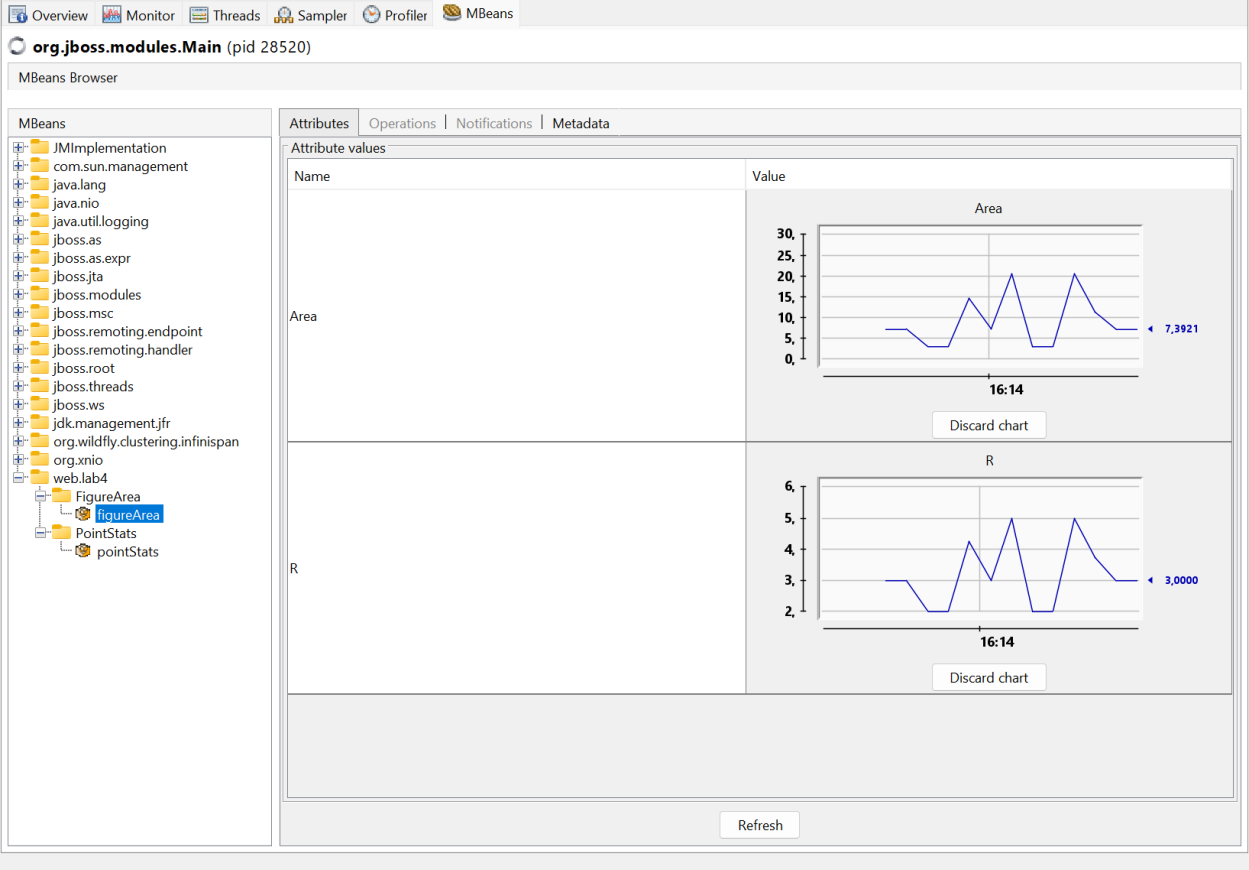
VisualVM

Графики

pointStats:



figureArea:



Поток, потребляющий наибольший процент времени CPU.

Overview

Monitor

Threads

Sampler

Profiler

MBeans

org.jboss.modules.Main (pid 22196)

Sampler

Settings

Samp...

CPU

Memory

Stop

Status: CPU sampling in progress

CPU samples

Thread CPU time

Results: Statistics: Threads Count: 107 Total Time (CPU): 57 515 ms

Thread Dump

Name	Thread Time (CPU)	Thread Time (CPU) / sec
pool-9-thread-2	19 296 ms (33.6 %)	46.9 ms (4.7 %)
management I/O-1	11 078 ms (19.3 %)	15.6 ms (1.6 %)
pool-9-thread-6	9 218 ms (16 %)	0.0 ms (0 %)
default task-3	3 296 ms (5.7 %)	0.0 ms (0 %)
JFR Periodic Tasks	2 906 ms (5.1 %)	0.0 ms (0 %)
MSC service thread 1-4	1 421 ms (2.5 %)	0.0 ms (0 %)
DestroyJavaVM	1 406 ms (2.4 %)	0.0 ms (0 %)
MSC service thread 1-8	1 234 ms (2.1 %)	0.0 ms (0 %)
MSC service thread 1-6	968 ms (1.7 %)	0.0 ms (0 %)
MSC service thread 1-2	937 ms (1.6 %)	0.0 ms (0 %)
MSC service thread 1-3	703 ms (1.2 %)	0.0 ms (0 %)
management task-2	671 ms (1.2 %)	0.0 ms (0 %)
MSC service thread 1-7	593 ms (1 %)	0.0 ms (0 %)
DeploymentScanner-threads - 2	515 ms (0.9 %)	0.0 ms (0 %)
ServerService Thread Pool -- 5	500 ms (0.9 %)	0.0 ms (0 %)
MSC service thread 1-5	453 ms (0.8 %)	0.0 ms (0 %)
Mojarra-WebResourceMonitor-1-thread-1	406 ms (0.7 %)	0.0 ms (0 %)
MSC service thread 1-1	390 ms (0.7 %)	0.0 ms (0 %)
management task-1	375 ms (0.7 %)	0.0 ms (0 %)
Attach Listener	281 ms (0.5 %)	0.0 ms (0 %)
DeploymentScanner-threads - 1	93.7 ms (0.2 %)	0.0 ms (0 %)
Weld Thread Pool -- 10	93.7 ms (0.2 %)	0.0 ms (0 %)
default I/O-26	78.1 ms (0.1 %)	0.0 ms (0 %)
jaeger.RemoteReporter-FlushTimer	78.1 ms (0.1 %)	0.0 ms (0 %)
Weld Thread Pool -- 8	78.1 ms (0.1 %)	0.0 ms (0 %)
default I/O-3	62.5 ms (0.1 %)	0.0 ms (0 %)
ServerDeploymentRepository-temp-threads - 1	46.8 ms (0.1 %)	0.0 ms (0 %)
Weld Thread Pool -- 11	46.8 ms (0.1 %)	0.0 ms (0 %)
Weld Thread Pool -- 17	46.8 ms (0.1 %)	0.0 ms (0 %)
Reference Handler	15.6 ms (0 %)	0.0 ms (0 %)
IdleRemover	15.6 ms (0 %)	0.0 ms (0 %)
management Accept	15.6 ms (0 %)	0.0 ms (0 %)
default I/O-6	15.6 ms (0 %)	0.0 ms (0 %)

В ходе профилирования был использован режим Sampler → Thread CPU time. После запуска sampling и выполнения действий в веб-приложении наибольший процент процессорного времени потреблял поток pool-9-thread-2. Его суммарное CPU-время составило 19 296 мс, что соответствует 33.6% от общего CPU-времени.

Проблема

В ходе профилирования приложения была обнаружена проблема неэффективного использования памяти. Класс ResultsBean имел область видимости @SessionScoped и хранил в поле history полный список результатов:

```
public class ResultsBean implements Serializable {
    private List<HitResult> history;

    @PostConstruct
    public void init() {
        history = new ArrayList<>(repository.findAll());
    }
}

public class HitRepository {
    public List<HitResult> findAll() {
        EntityManager em = em();
        try {
            return em.createQuery(
```

```

        "select h from HitResult h order by h.at desc",
        HitResult.class
    ).getResultList();
} finally {
    em.close();
}
}
}

```

Так как ResultsBean является сессионным bean-компонентом, для каждой пользовательской сессии создавался отдельный экземпляр ResultsBean, который хранил собственную копию всей истории. В результате при увеличении количества активных сессий росло количество объектов HitResult, удерживаемых в heap-памяти.

Как была найдена проблема?

Для локализации проблемы использовались VisualVM и утилита jcmd. В браузере с помощью скрипта создавалось большое количество точек (2000-3000). После чего открыты дополнительные пользовательские сессии через вкладки браузера и curl:

```

1..10 | ForEach-Object {
    $cookie = "session-$_txt"
    curl.exe -s -c $cookie -b $cookie "http://127.0.0.1:19080/lab3/main.xhtml" -o "page-$_html"
    Write-Host "Created session $_"
    Start-Sleep -Seconds 1
}

```

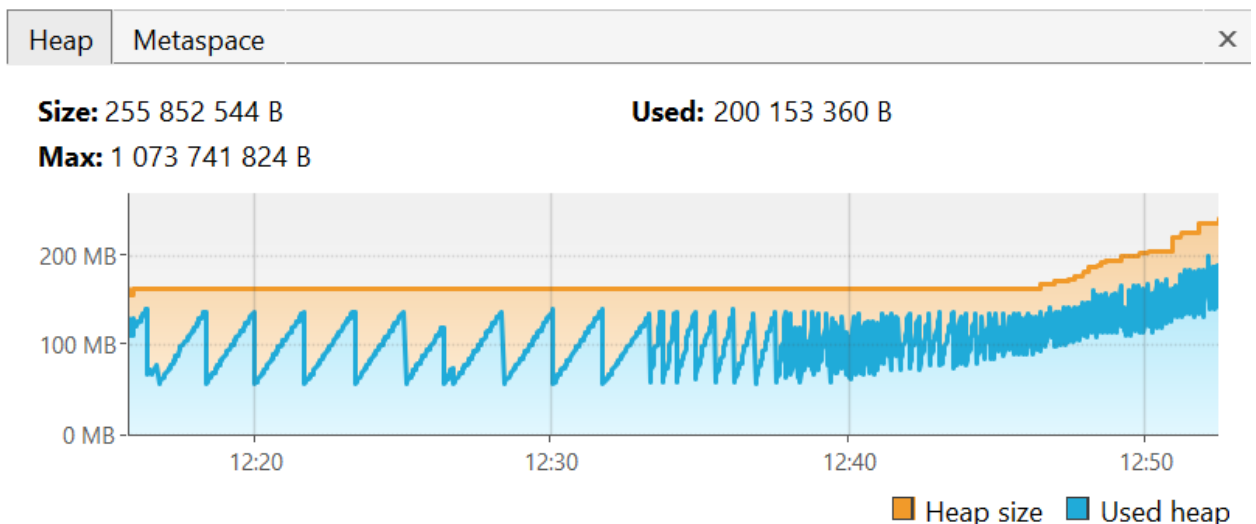
На графике VisualVM наблюдался рост используемой памяти. Для уточнения причины был выполнен анализ heap через GC.class_histogram:

```

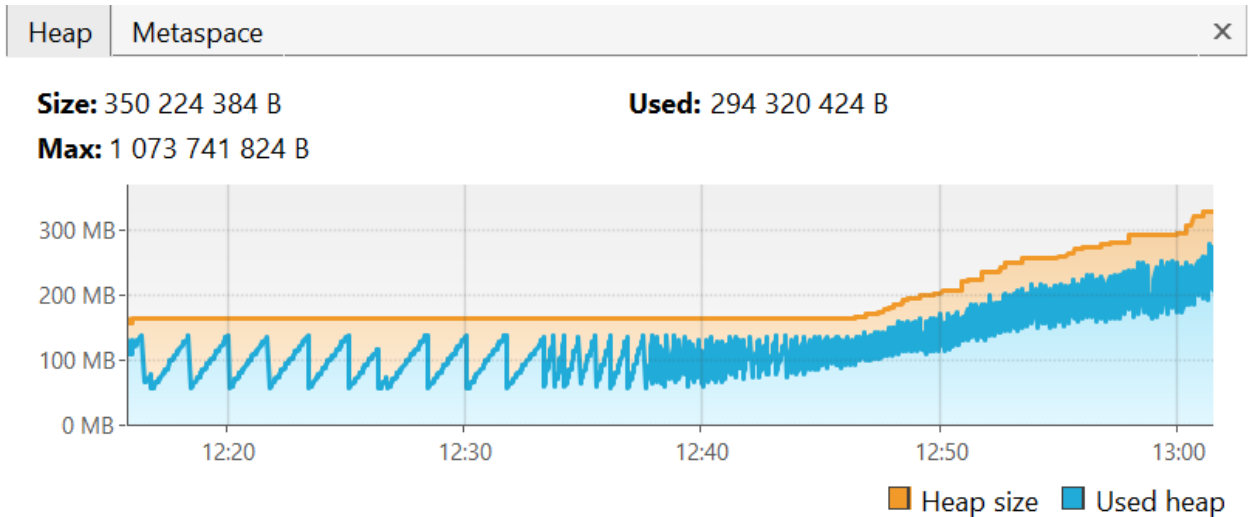
PID=$(/usr/local/openjdk17/bin/jcmd -l | grep -E "jboss-
modules|org.jboss.as.standalone|org.jboss.modules.Main" | head -1 | awk '{print $1}')
echo $PID
/usr/local/openjdk17/bin/jcmd $PID GC.run
/usr/local/openjdk17/bin/jcmd $PID GC.class_histogram > ~/opi4/hist-after-sessions.txt
grep -E "ResultsBean|HitResult|ArrayList|Ljva.lang.Object;" ~/opi4/hist-after-sessions.txt |
head -40

```

После 10 сессий:



После 20 сессий:



Jcmd:

```
[s465142@helios ~]$ PID=$( /usr/local/openjdk17/bin/jcmd -l | grep -E "jboss-modules|org.jboss.as.standalone|org.jboss.modules.Main" | head -1 | awk '{print $1}')
echo $PID
/usr/local/openjdk17/bin/jcmd $PID GC.run
/usr/local/openjdk17/bin/jcmd $PID GC.class_histogram > ~/opi4/hist-after-sessions.txt
grep -E "ResultsBean|HitResult|ArrayList|\\[Ljava.lang.Object;" ~/opi4/hist-after-sessions.txt | head -40
Picked up _JAVA_OPTIONS: -XX:MaxHeapSize=1G -XX:MaxMetaspaceSize=128m
47562
Picked up _JAVA_OPTIONS: -XX:MaxHeapSize=1G -XX:MaxMetaspaceSize=128m
47562:
Command executed successfully
Picked up _JAVA_OPTIONS: -XX:MaxHeapSize=1G -XX:MaxMetaspaceSize=128m
 1:      1464147      51365504 [Ljava.lang.Object; (java.base@17.0.12)
 6:      391513      9396312  java.util.ArrayList (java.base@17.0.12)
 7:      181343      8704464  web.models.HitResult
131:       1360       32640   java.util.concurrent.CopyOnWriteArrayList (java.base@17.0.12)
134:       1330       31920   java.util.Arrays$ArrayList (java.base@17.0.12)
517:        122        2928   org.jboss.weld.util.collections.ImmutableArrayList
603:         69        2208   java.util.ArrayList$Itr (java.base@17.0.12)
1011:        24         768   web.models.ResultsBean
1592:         2         304   [[Ljava.lang.Object; (java.base@17.0.12)
4908:         1         48   web.models.ResultsBean$Proxy$_$$_WeldClientProxy
6817:         1         24   java.util.concurrent.CopyOnWriteArrayList$COWIterator (java.base@17.0.12)
10288:        1         16   org.infinispan.protostream.types.java.collections.ArrayListAdapter
10289:        1         16   org.infinispan.protostream.types.java.collections.ArrayListAdapter$__Marshaller_64
4b926b0d3f3b832ca16b720ed12a15fdeccbf32741bf201b2c4bb4245d40c
10650:        1         16   org.jboss.as.connector.util.CopyOnWriteArrayListMultiMap
12808:        1         16   web.models.HitResult$HibernateAccessOptimizer$aGLOVSU
12809:        1         16   web.models.HitResult$HibernateInstantiator$YnIUSu3W
[s465142@helios ~]$
```

Как была исправлена проблема?

Из ResultsBean было удалено хранение полного списка результатов. Вместо списка history в сессии теперь хранится только состояние пагинации: номер текущей страницы и размер страницы.

Текущая страница результатов загружается из базы данных по запросу:

```
public class ResultsBean implements Serializable {

    public List<HitResult> getPage() {
        normalizePageIndex();
        if (repository.count() <= 0) {return Collections.emptyList();}
        return repository.findPage(pageIndex, pageSize);
    }
}
```

В класс HitRepository были добавлены методы для постраничной загрузки и подсчёта общего количества записей:

```
public List<HitResult> findPage(int pageIndex, int pageSize) {
    EntityManager em = em();
    try {
        int safePageIndex = Math.max(0, pageIndex);
        int safePageSize = Math.max(1, pageSize);

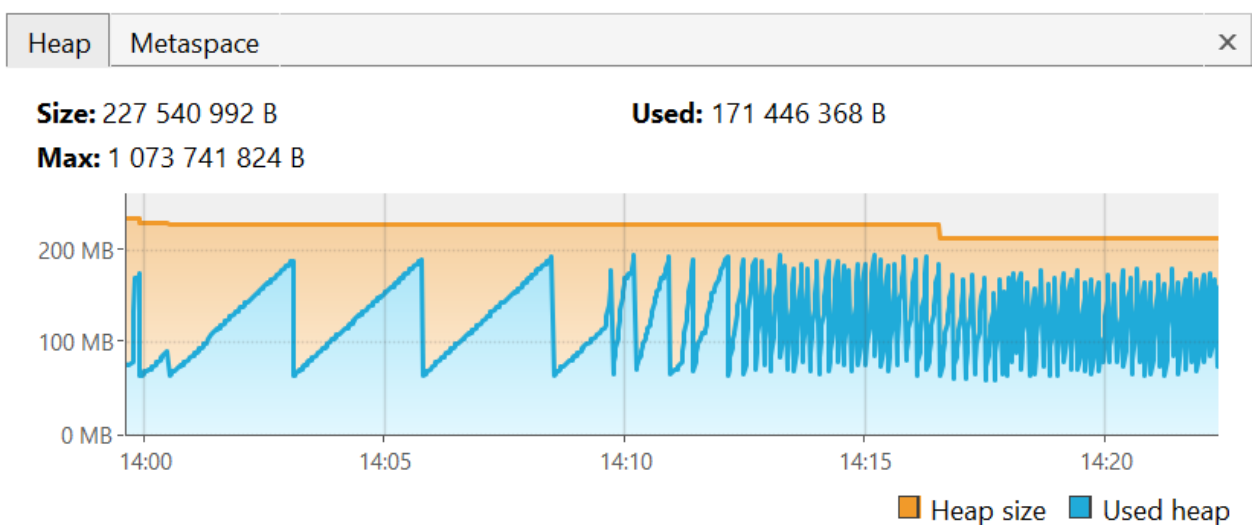
        return em.createQuery(
            "select h from HitResult h order by h.at desc",
            HitResult.class
        ).setFirstResult(safePageIndex * safePageSize)
        .setMaxResults(safePageSize)
        .getResultList();
    } finally {
        em.close();
    }
}
```

После добавления новой точки объект сохраняется в базе данных, а ResultsBean не добавляет его в большой список в памяти:

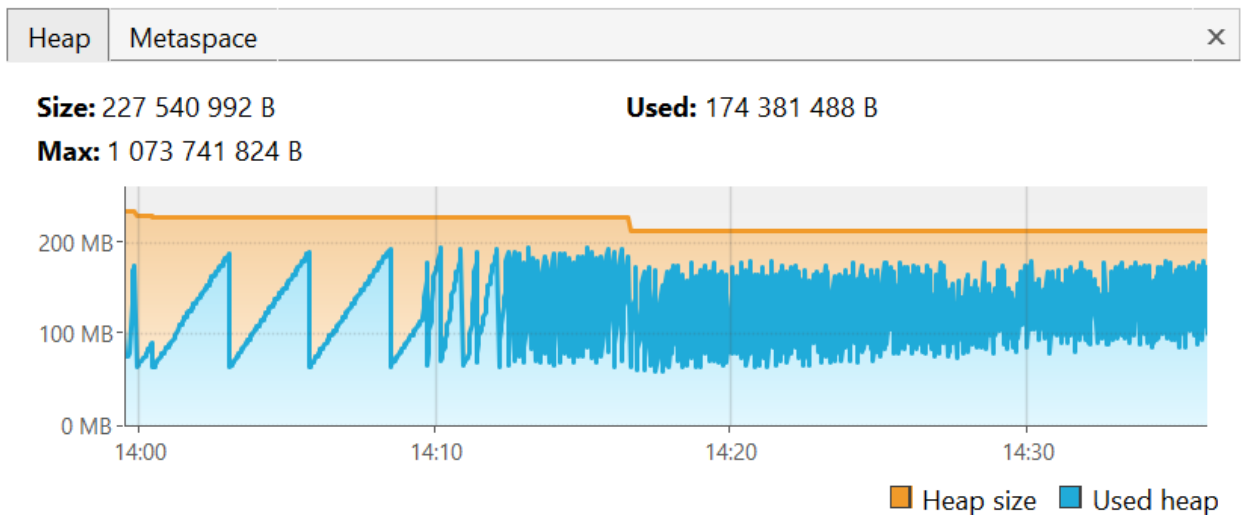
```
public void add(HitResult r) {
    repository.save(r);
    monitoringService.onPointAdded(r);
    pageIndex = 0;
}
```

После внесения исправлений был повторен тот же порядок действий

После 10 сессий:



После 20 сессий:



Jcmd:

```
[s465142@helios ~]$ PID=$(/usr/local/openjdk17/bin/jcmd -l | grep -E "jboss-modules|org.jboss.as.standalone|org.jboss.modules.Main" | head -1 | awk '{print $1}')
echo $PID
/usr/local/openjdk17/bin/jcmd $PID GC.run
/usr/local/openjdk17/bin/jcmd $PID GC.class_histogram > ~/opi4/hist-after-sessions.txt
grep -E "ResultsBean|HitResult|ArrayList|Ljjava.lang.Object;" ~/opi4/hist-after-sessions.txt | head -40
Picked up _JAVA_OPTIONS: -XX:MaxHeapSize=1G -XX:MaxMetaspaceSize=128m
75843
Picked up _JAVA_OPTIONS: -XX:MaxHeapSize=1G -XX:MaxMetaspaceSize=128m
75843:
Command executed successfully
Picked up _JAVA_OPTIONS: -XX:MaxHeapSize=1G -XX:MaxMetaspaceSize=128m
3: 92421 4384008 [Ljava.lang.Object; (java.base@17.0.12)
9: 37945 910680 java.util.ArrayList (java.base@17.0.12)
124: 1337 32088 java.util.concurrent.CopyOnWriteArrayList (java.base@17.0.12)
126: 1329 31896 java.util.Arrays$ArrayList (java.base@17.0.12)
519: 122 2928 org.jboss.weld.util.collections.ImmutableArrayList
606: 69 2208 java.util.ArrayList$Itr (java.base@17.0.12)
872: 33 1056 web.models.ResultsBean
1590: 2 304 [Ljava.lang.Object; (java.base@17.0.12)
4922: 1 48 web.models.ResultsBean$Proxy$_$$_WeldClientProxy
6828: 1 24 java.util.concurrent.CopyOnWriteArrayList$COWIterator (java.base@17.0.12)
10105: 1 16 org.infinispan.protostream.types.java.collections.ArrayListAdapter
10106: 1 16 org.infinispan.protostream.types.java.collections.ArrayListAdapter$___Marshaller_64
4b926b0d3f3b832ca16b720ed12a15fdedccbf32741bf201b2c4bb4245d40c
10467: 1 16 org.jboss.as.connector.util.CopyOnWriteArrayListMultiMap
12626: 1 16 web.models.HitResult$HibernateAccessOptimizer$Sx0xTlsU
12627: 1 16 web.models.HitResult$HibernateInstantiator$cRDJr50E
[s465142@helios ~]$
```

После исправления количество экземпляров ResultsBean всё ещё увеличивается вместе с количеством активных пользовательских сессий. Это является ожидаемым поведением для @SessionScoped bean. Однако объекты HitResult больше не удерживаются в памяти через ResultsBean, так как полный список истории был удалён из сессионного bean.

Вывод

В ходе лабораторной работы были реализованы MBean-классы для мониторинга веб-приложения. Для мониторинга и профилирования работы программы использовались JConsole и VisualVM. Также была выявлена проблема неэффективного использования памяти: ResultsBean хранил всю историю результатов в SessionScoped bean. Для исправления была реализована страничная загрузка результатов из базы данных. После повторного профилирования объём используемой heap-памяти уменьшился, следовательно, проблема была устранена.